

Inteligencia Artificial

Problemas de Optimización

Ing. González H. Alexis



Tabla de Contenidos

01

Active Recall

Repaso del tema anterior

02

Main Content

Algoritmos de búsqueda local
Gradiente descendiente o
ascendente
Simulated Annealing
Búsqueda local beam
Algoritmos genéticos
Búsqueda en espacios
continuos

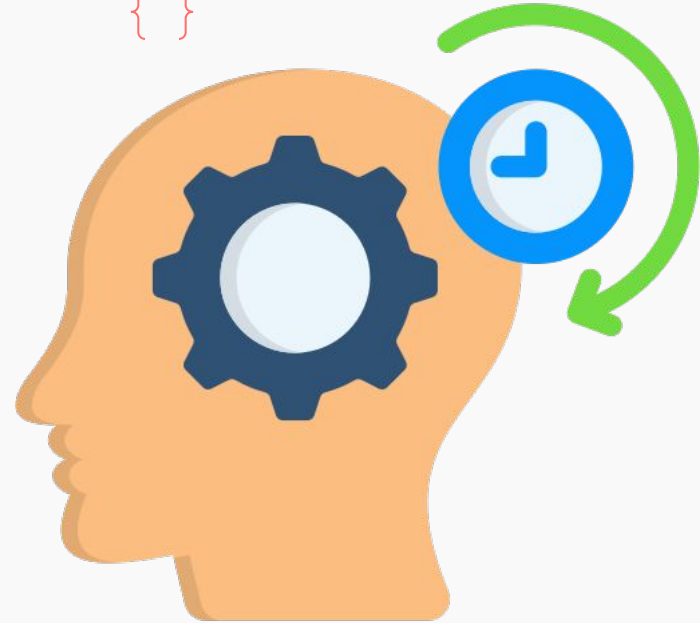
03

Live Coding Pair Programming

Ejercitación & Feedback

01

Active Recall



{ }

Algoritmos de Búsqueda: Cuadro Comparativo

Búsqueda No Informada (Ciega)

Búsqueda Primero en Anchura (BFS)

Explora nivel por nivel usando FIFO. Garantiza solución óptima si los costos de paso son iguales.

Costo Uniforme / Dijkstra

Expande el nodo con menor costo acumulado $g(n)$. Encuentra siempre la ruta con costo mínimo.

Búsqueda Primero en Profundidad (DFS)

Explora la rama más profunda usando LIFO. Muy eficiente en memoria, pero no garantiza optimalidad.

Profundidad Limitada (DLS)

Aplica DFS con un límite fijo de profundidad l . Evita bucles infinitos pero puede no hallar la solución.

Profundidad Iterativa (IDDFS)

Incrementa gradualmente el límite de DLS. Combina la eficiencia de memoria de DFS con la optimalidad de BFS.

Búsqueda Informada (Heurística)

Búsqueda Voraz (Greedy Best-First)

Función de evaluación: $f(n) = h(n)$

Expande el nodo que parece estar más cerca de la meta según la heurística $h(n)$. Rápido, pero no garantiza encontrar la solución más corta u óptima.

ALGORITMO DESTACADO / RECOMENDADO

Búsqueda A* (A-Estrella)

Función de evaluación: $f(n) = g(n) + h(n)$

Combina el costo real acumulado $g(n)$ y la estimación $h(n)$. Es óptimo y completo si la heurística $h(n)$ es admisible y consistente.

Criterio Clave de Selección

Sin Heurística: Usar IDDFS para memoria limitada o Dijkstra/BFS para costo óptimo sin información extra.

Con Heurística: Usar A* para optimalidad comprobada o Greedy para rapidez en grafos complejos.



02

Main Content { }

Algoritmos de Búsqueda Local

Comparativa de enfoques en espacios de búsqueda

ENFOQUE ANTERIOR / TRADICIONAL



Búsqueda Sistemática

Premisa: El camino recorrido es la solución

- Diseñada para explorar sistemáticamente espacios de búsqueda.
- Mantiene uno o más caminos en memoria durante el proceso.
- Registra minuciosamente qué alternativas se han explorado y cuáles no.
- Al hallar la meta, el **camino recorrido** constituye la solución al problema.

NUEVO PARADIGMA



Búsqueda Local

Premisa: Solo importa la configuración final

- Aplica a problemas donde el camino exacto no tiene relevancia.
- El objetivo es encontrar una **configuración de estado válida** o de costo óptimo.
- Opera evaluando y modificando la configuración actual.

Caso Práctico: Resolución de Sudokus



No importa la secuencia de pasos empleada para colocar los números, únicamente interesa obtener el tablero final resuelto.

Algoritmos de Búsqueda Local

FUNCIONAMIENTO GENERAL



Búsqueda Orientada a Estados

Si **no importa el camino** al objetivo, podemos considerar algoritmos que no guardan el historial de rutas.

Funcionan evaluando exclusivamente un **solo estado actual** y se desplazan únicamente hacia los **vecinos** de dicho estado.

VENTAJA 01



Uso Mínimo de Memoria

Al no requerir el almacenamiento de los caminos ni del árbol de búsqueda explícito, consumen **muy poca memoria** durante la ejecución.

VENTAJA 02



Gran Escalabilidad

Pueden encontrar **soluciones razonables** en espacios de estado **enormes, extremadamente complejos o infinitos (continuos)**.

Algoritmos de Búsqueda Local

Problemas de optimización

CONCEPTO FUNDAMENTAL



Búsqueda del Estado Óptimo

Útiles para resolver problemas de optimización puros, donde el objetivo es **encontrar el mejor estado** según una función objetivo. Consiste en **minimizar o maximizar** el valor de una variable o función.

Optimización Discreta



Variables Discretas

- La búsqueda se realiza dentro de un **conjunto contable**.
- El objeto a hallar puede ser un **número entero**, una **permutación** o un **grafo**.

Optimización Continua



Variables Continuas

- El objetivo es encontrar un **valor óptimo** de una función continua.
- Opera sobre espacios de estados y variables de **dominio continuo**.

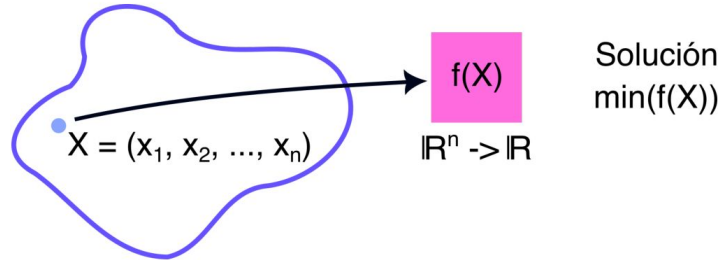
Algoritmos de Búsqueda Local

Problemas de optimización

FORMULACIÓN DEL PROBLEMA

Σ Espacio de Estados y Mapeo

Representación matemática básica

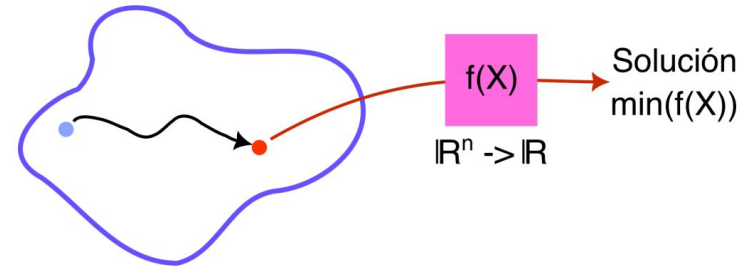


- Estado representado por $\mathbf{X} = (x_1, x_2, \dots, x_n)$.
- Función de evaluación $f(\mathbf{X}): \mathbb{R}^n \rightarrow \mathbb{R}$.
- Objetivo: Encontrar la solución $\min(f(\mathbf{X}))$.

PROCESO DE OPTIMIZACIÓN

Navegación Local de Estados

Búsqueda iterativa por vecindad



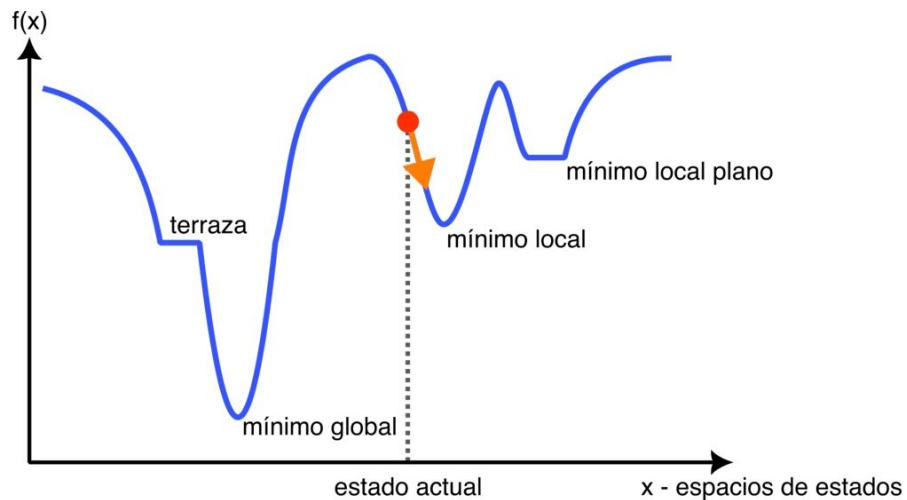
- Inicia en un estado e explora **vecinos inmediatos**.
- Se desplaza hacia estados que **mejoren** $f(\mathbf{X})$.
- Detiene la búsqueda al alcanzar un **óptimo local**.

Algoritmos de Búsqueda Local

Problemas de optimización

VISUALIZACIÓN DEL ESPACIO DE ESTADOS

En un caso de una sola variable $f(x)$, podemos identificar las siguientes estructuras:



ELEMENTOS CLAVE DEL PAISAJE

Estado Actual

Punto de partida o posición evaluada en la iteración actual.

Mínimo Global vs. Local

El **mínimo global** es el objetivo óptimo, mientras que un **mínimo local** representa una trampa donde los vecinos inmediatos no ofrecen mejora.

Terrazas y Mínimos Planos

Regiones donde la función de evaluación permanece constante, dificultando la guía del gradiente.

Algoritmos de Búsqueda Local

Problemas de optimización

DOMINIO DISCRETO



Espacio Discreto

Estados finitos o contables

Algoritmos aplicables a este espacio:

- Gradiente descendiente o ascendente
- Simulated annealing
- Búsqueda Local Beam
- Algoritmos genéticos

DOMINIO CONTINUO



Espacio Continuo

Variables sobre dominios infinitos/reales

Algoritmos aplicables a este espacio:

- Gradiente descendiente o ascendente en espacio continuo

Gradiente Descendiente o Ascendente Discreto

Problemas de optimización

FUNCIONAMIENTO Y REGLAS

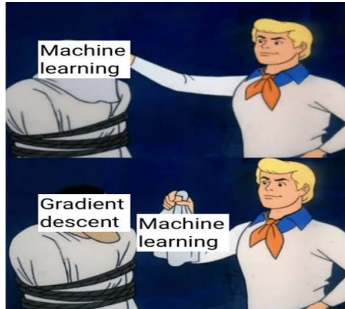
Este algoritmo continuamente se mueve en dirección de mayor valor decreciente o creciente.

Criterio de Parada

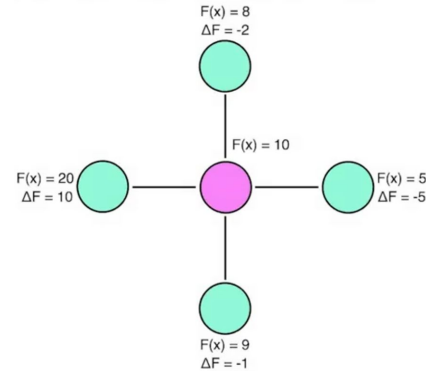
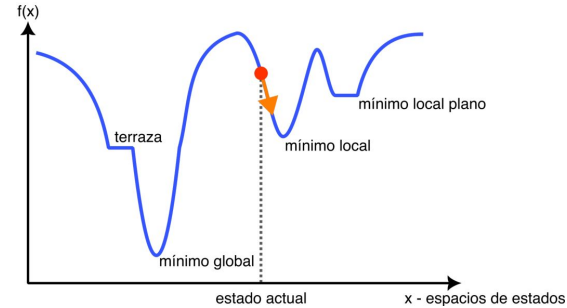
Termina la búsqueda en donde ningún vecino está más bajo (o más alto, según el objetivo).

Evaluación Local

Este algoritmo no mira más allá de lo que tiene en sus vecinos inmediatos.



VISUALIZACIÓN DEL PAISAJE DE BÚSQUEDA



Gradiente Descendiente o Ascendente Discreto

Problemas de optimización

CONCEPTO GENERAL

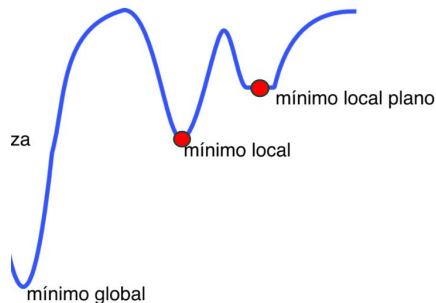
Esta búsqueda se le llama **búsqueda local voraz (greedy)**, porque avanza a un estado vecino sin pensar en dónde ir después. Avanza muy rápido hacia una solución, pero es fácil llegar a un estado no ideal. Se atasca por los siguientes motivos:

OBSTÁCULO 01



Mínimo (máximo) local

Un mínimo local es un valle que es más bajo que sus estados vecinos, pero estás más arriba que el mínimo global.



OBSTÁCULO 02



Meseta

Una meseta es un área donde la función de evaluación es plana.

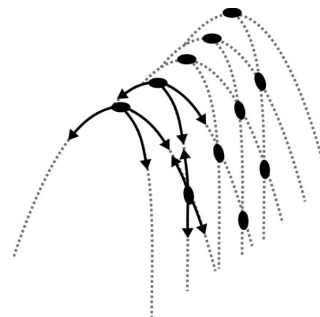


OBSTÁCULO 03



Crestas

Las crestas causan una secuencia de mínimos locales que hace muy difícil la navegación.



Gradiente Descendiente o Ascendente Discreto

Problemas de optimización

VARIANTE 01



Estocástico

Se escoge aleatoriamente la dirección de entre los movimientos ascendentes posibles.

Característica:

Converge más lento, pero en ciertos casos encuentra mejor solución.

VARIANTE 02



Primera Opción

Posee una planificación la cual genera sucesores de forma estocástica al azar.

Criterio:

Evalúa sucesivamente hasta generar un estado que sea mejor que el estado actual.

VARIANTE 03



Reinicio Aleatorio

Conduce a una serie de búsquedas en ascensión de colinas.

Funcionamiento:

Inicia desde estados generados aleatoriamente, deteniéndose al encontrar un objetivo.

Simulated Annealing

Problemas de optimización

DISTRIBUCIÓN DE BOLTZMANN

Para determinar si se realiza el cambio de estado, se hace uso de la **distribución de Boltzmann**, una distribución de probabilidad de partículas en un sistema a través de varios estados posibles.

EXPRESIÓN MATEMÁTICA

Σ

Probabilidad de Aceptación

$$F \propto e^{-\frac{\Delta E}{kT}}$$

REGLA DE DECISIÓN Y COMPORTAMIENTO



Criterio de Selección

- Diferencia de Energía / Costo (ΔE):

Cuanto mayor sea la diferencia de energía entre estados, **menos probable** es que se elija el cambio.

- Efecto de la Temperatura (T):

Si la temperatura es **alta**, la probabilidad aumenta, facilitando que de todas formas se elija el estado propuesto.

Búsqueda Local Beam

Problemas de optimización

CONCEPTO BASE



Memoria de k Estados

Buscar desde un único estado inicial es una medida extrema de ahorro de memoria.

Estrategia:

Guarda la información de k estados inicializados al azar y busca sobre ellos.

PASO A PASO



Generación y Selección

En cada paso del proceso se generan los sucesores de los k estados.

Criterio:

Si ninguno cumple el objetivo, se seleccionan los k mejores sucesores de la lista global.

VENTAJA CLAVE



Pase de Información

A diferencia de ejecutar k búsquedas paralelas aisladas, aquí existe **intercambio de información**.

Efecto:

Abandona rápidamente las búsquedas infructuosas y mueve sus recursos hacia el mayor progreso.

Búsqueda Local Beam

Problemas de optimización

LIMITACIÓN



Carencia de Diversidad

El principal inconveniente es que puede sufrir **falta de diversidad** de estados.

Efecto:

El algoritmo corre el riesgo de enfocarse en una sola región muy limitada.

VARIANTE ESTOCÁSTICA



Haz Estocástico

En vez de elegir estrictamente a los **k mejores sucesores**, se eligen aleatoriamente.

Mecanismo:

Utiliza una función de temperatura similar a **Simulated Annealing**.

INSPIRACIÓN BIOLÓGICA



Selección Natural

Muestra un claro parecido con el proceso de **selección natural**.

Supervivencia:

Los descendientes pueblan la siguiente generación según su valor de **idoneidad o salud**.

Algoritmos Genéticos

Problemas de optimización

CONCEPTO BASE



Reproducción Padres

Variante de la **búsqueda local beam estocástica**.

Mecanismo:

Los sucesores se generan combinando **dos estados padres** (reproducción), en lugar de alterar un solo estado.

INSPIRACIÓN BIOLÓGICA



Selección Natural

Mantiene la **analogía directa** con el proceso de evolución biológica.

Principio:

La combinación de información entre individuos sigue la misma lógica de **selección natural**.

ESTRUCTURA DE DATOS

Población y Código

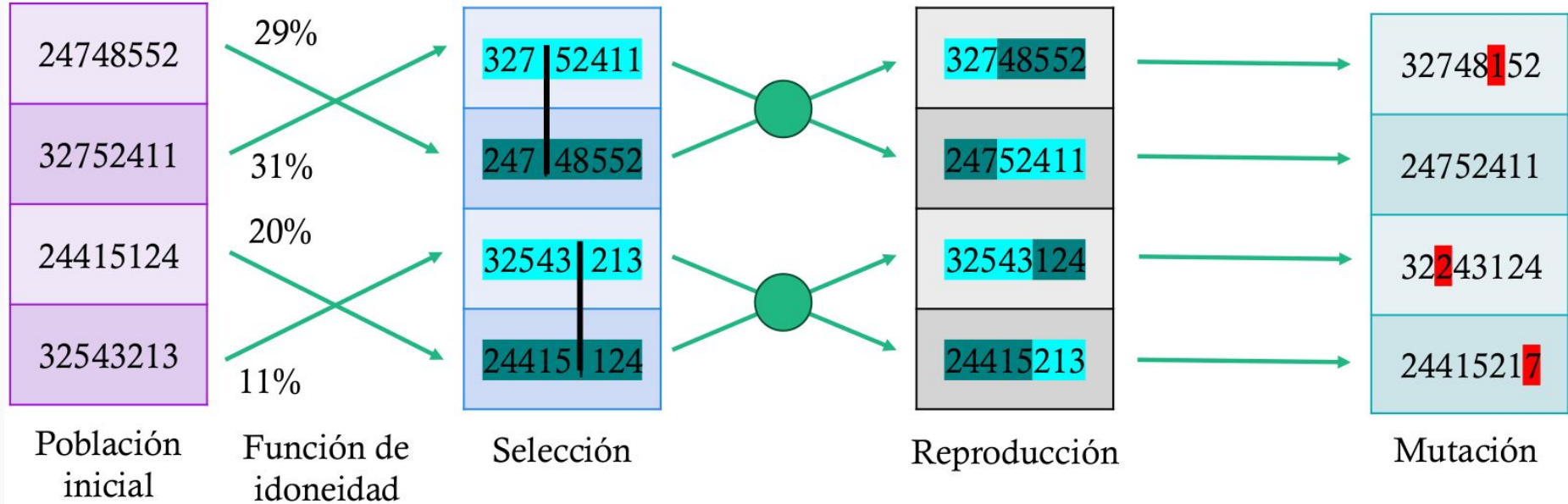
Comienzan con un conjunto de **k estados aleatorios**, denominado **población**.

Individuos:

Cada estado se representa como una **cadena de caracteres** sobre un alfabeto finito (**código genético**).

Algoritmos Genéticos

Problemas de optimización



Algoritmos Genéticos

Problemas de optimización



Exploración y Tendencia

Combina una **tendencia ascendente** con exploración aleatoria e intercambio de información entre hilos paralelos.



Operación de Cruce (Crossover)

Su principal ventaja radica en **combinar bloques de letras** que han evolucionado de forma independiente.



Mayor Granularidad

Aumenta sensiblemente el **nivel de granularidad** en el que opera la búsqueda para construir soluciones útiles.

```
def gen_alg(poblacion, fun_id):
    new_poblacion = []

    while 1:
        for i in range(len(poblacion)):
            x = sel_aleatory(poblacion, fun_id)
            y = sel_aleatory(poblacion, fun_id)
            child = reproduction(x, y)
            dice = random()
            if dice < 0.01:
                child = mutate(child)
            new_poblacion.append(child)
        poblacion = new_poblacion

        for z in poblacion:
            if z is solution:
                return z

def reproduction(x, y):
    n = len(x)
    c = random_int(0, n)
    return x[:c] + y[c:]
```

Búsqueda en Espacios Continuos



Concepto e Historia

Desafío del Espacio Continuo

Los algoritmos discretos tradicionales no son aplicables directamente en espacios continuos, ya que la cantidad de sucesores posibles es **infinita**.

Origen Matemático

La literatura sobre búsqueda en espacios continuos es muy amplia y se remonta a la época de **Newton y Leibniz** en el siglo XVII.



Ejemplo: Ubicación de Aeropuertos

Se requiere posicionar **2 nuevos aeropuertos** en Argentina. El objetivo es minimizar la distancia al cuadrado respecto a 4 grandes ciudades: **Buenos Aires, La Plata, Córdoba y Rosario**.



Espacio de Estados (4D)

Definido por **4 variables continuas**: (x_1, y_1) y (x_2, y_2) para las coordenadas de los aeropuertos. Desplazarse en este espacio equivale a mover los aeropuertos.



Función Objetivo

Se evalúa como **$f(x_1, y_1, x_2, y_2)$** calculando fácilmente la suma de las **distancias euclidianas** entre los aeropuertos y cada ciudad.

Búsqueda en Espacios Continuos



Función de Costo

Expresión Matemática

$$f(x_1, y_1, x_2, y_2)$$

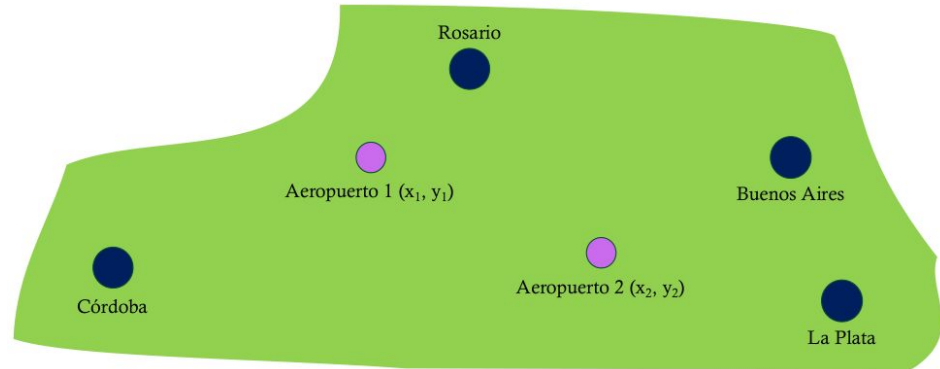
Criterio de Optimización

Suma de las distancias euclidianas entre las ubicaciones continuas de los aeropuertos y las ciudades principales.

Variables Continuas

- Coordenadas Aeropuerto 1: (x_1, y_1)
- Coordenadas Aeropuerto 2: (x_2, y_2)

Representación del Espacio de Búsqueda



Búsqueda en Espacios Continuos



Función de Costo

Expresión Matemática

$$f(x_1, y_1, x_2, y_2)$$

Criterio de Optimización

Suma de las distancias euclidianas entre las ubicaciones continuas de los aeropuertos y las ciudades principales.

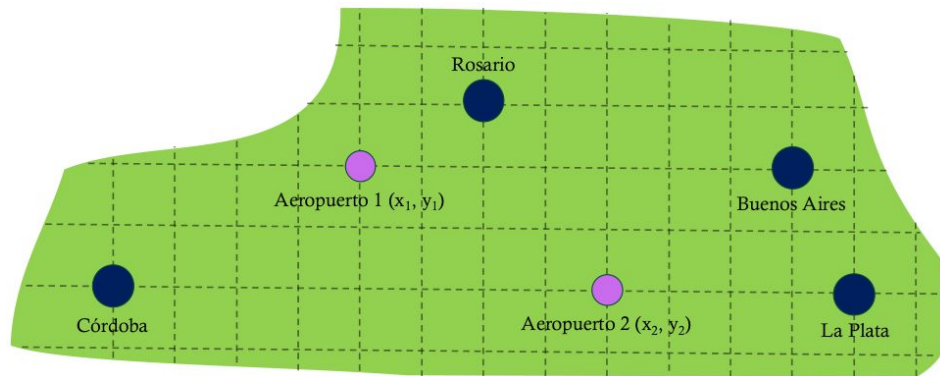
Variables Continuas

- Coordenadas Aeropuerto 1: (x_1, y_1)
- Coordenadas Aeropuerto 2: (x_2, y_2)

Discretizar el Espacio

Una forma de resolver la búsqueda continua es discretizar el espacio en posiciones o estados finitos.

Representación del Espacio de Búsqueda



Búsqueda en Espacios Continuos



Optimización por Gradiente

Hay métodos de usar el gradiente para encontrar un máximo o mínimo:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2} \right)$$



Resolución Analítica

En algunos casos podemos resolver la ecuación diferencial:

$$\nabla f = 0$$

Caso de un solo aeropuerto: La solución corresponde a la media aritmética de la ubicación de las ciudades.



Búsqueda en Espacios Continuos



Algoritmo de Gradiente

En muchos casos esto no puede resolverse directamente. Hacemos uso del algoritmo de gradiente realizando un cambio local:

$$X_{new} = X - \alpha \nabla f(X)$$



Consideraciones Clave

Tasa de Aprendizaje (α): Constante pequeña, conocida en redes neuronales como constante de aprendizaje.

Gradiente Empírico: Si no hay función diferenciable, se evalúan pequeños incrementos y decrementos en cada coordenada.



Búsqueda en Espacios Continuos



Importancia de la Tasa de Aprendizaje (α)

La elección de α es sumamente importante: determina la magnitud del paso en cada iteración. Una elección incorrecta impide encontrar el mínimo deseado.



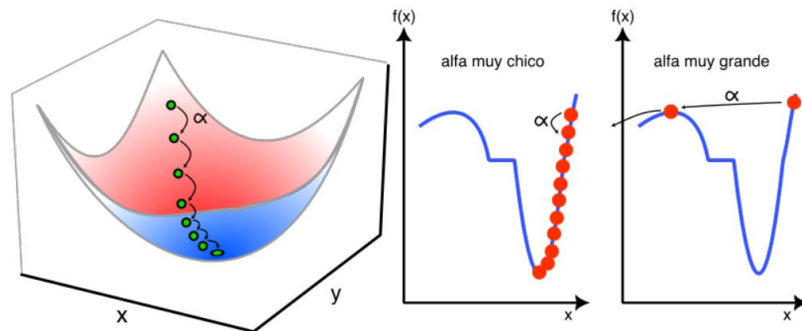
α Muy Pequeña

- Avance extremadamente lento.
- Se requieren demasiados pasos para aproximar la solución.



α Muy Grande

- Pasos excesivamente largos sobrepasan el mínimo.
- Provoca oscilaciones y el algoritmo nunca converge.





Google Colab

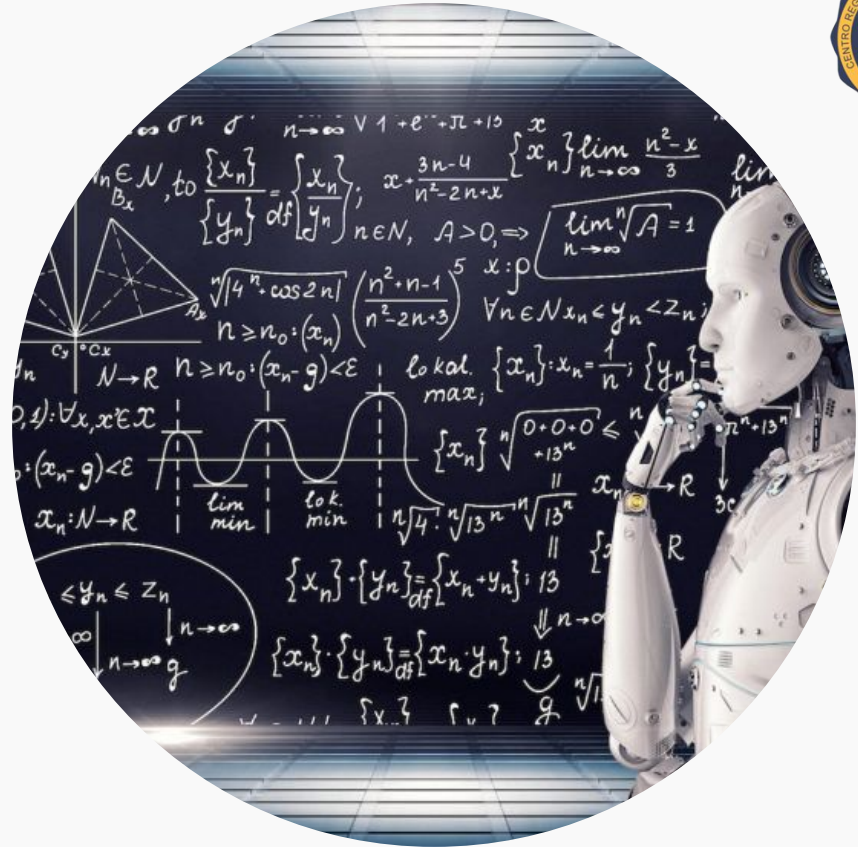
Ejemplo en Python

08_optimizacion



¡Gracias!

¿Consultas?



Ing. González H. Alexis